

GHAW-H: A Dataset of GitHub Agentic Workflow Histories

Pablo Valenzuela^{*†}, Timo Kehrer^{*}, and Sebastiano Panichella^{*‡}

^{*}University of Bern, Switzerland

[†]Universidad de La Frontera, Chile

[‡]Italian Institute of Artificial Intelligence for Industry (AI4I), Italy

Abstract—GitHub Agentic Workflows (GH-AW) are an emerging form of repository automation in which developers define agentic workflows in Markdown and execute them through GitHub Actions. However, GH-AW only became publicly available as a Technical Preview in February 2026, and little is known about its early adoption and use in public repositories. To support the systematic study of this emerging practice, we introduce *GHAW-H*, a dataset covering 264 early adopters of GH-AW in public GitHub repositories. The dataset links source Markdown files with their compiled lock files across version histories, providing a foundation for studying how natural-language agentic workflows are defined and evolve in software projects.

Index Terms—GitHub Agentic Workflows, GitHub Actions, agentic automation, natural-language software artifacts, dataset

I. HIGH-LEVEL OVERVIEW

Agentic Workflows are a form of automation that uses Large Language Models (LLMs) to interpret goals, use external tools, and perform actions in their environment [1]. Early LLM-based coding tools focused primarily on assisting developers with code completion and generation. In contrast, recent coding agents can interact directly with repositories and development tools to perform a broader range of software engineering activities [2], [3]. GitHub Agentic Workflows¹ (GH-AW) brings these capabilities to GitHub, enabling developers to define in Markdown how an agent should respond to repository events and execute tasks through GitHub Actions. However, the tool was released only recently, on February 13, 2026, and little is known about its adoption and use.

Figure 1 illustrates how a GH-AW agentic workflow is defined, compiled, and executed within a GitHub repository. In this example, the workflow definition is a Markdown file that combines the configuration in the *frontmatter* with task instructions expressed in natural language. GH-AW compiles this definition into a *lock* file, which runs as a GitHub Actions workflow. During execution, the agent interprets the instructions, analyzes the repository context, and performs the defined task. The workflow may produce observable outcomes

through *safe outputs*, such as adding labels, posting comments, or opening a pull request.

To support the systematic study of the early adoption of GH-AW, we introduce *GHAW-H*, a dataset built from 262 repositories that use this tool. *GHAW-H* includes the histories of 604 Markdown files and 2,820 observed versions. Each version preserves the original specification, including the literal *frontmatter*, the natural-language task instructions, and the corresponding compiled snapshot. It also includes the commit SHA, timestamp, position of the version within the file history, and repository-level metadata.

II. INTERNAL STRUCTURE

Table I summarizes the structure of *GHAW-H*, including its five normalized tables, record counts, and the GitHub evidence represented in each table. The dataset contains the complete histories of GH-AW files collected from 262 public repositories. These repositories used the tool between its release as a Technical Preview on February 13, 2026, and June 6, 2026, and represent its early adopters [4]. It comprises 604 source Markdown file histories and 2,820 versions. Each version is linked to repository-level metadata. Its representation includes a snapshot of the source Markdown file, a version record containing its commit SHA, timestamp, and position within the file history, and the corresponding compiled lock file snapshot recovered from the same commit.

III. HOW TO ACCESS

The *GHAW-H* dataset is publicly available on Hugging Face, where its five Parquet tables can be downloaded and inspected. The release includes four reproducible Jupyter notebooks covering basic dataset loading, general summaries, repository language analysis, and the analysis of source-file evolution histories. These notebooks load the published Parquet files directly from Hugging Face and can be executed through ready-to-use Google Colab links available in the GitHub repository and the Hugging Face dataset card. The release also includes the dataset schema, the table dictionary, and

¹<https://github.com/gh-aw/>

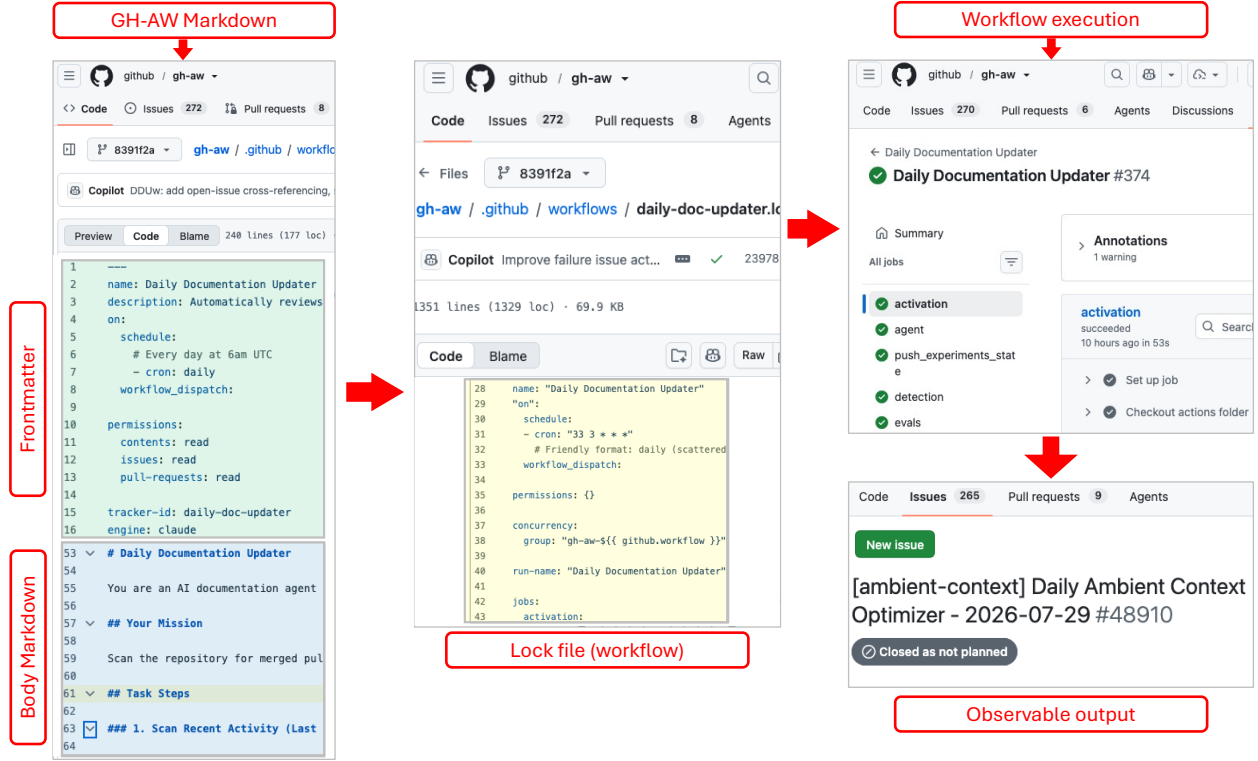


Fig. 1. Overview of a GitHub Agentic Workflow in a repository. A source Markdown file under `.github/workflows/` defines the workflow configuration (Frontmatter) and task instructions (Body Markdown). The GH-AW tooling compiles this source file into a Lock file, which can be executed through GitHub Actions. The execution may produce observable safe outputs, such as a repository comment or a new issue.

TABLE I
OVERVIEW OF THE GH-AW HISTORIES DATASET

	Table	# Records	Content
Repository Metadata	repository	262	Repository-level metadata (name, owner, URL, license, language, stars, forks, and timestamps)
	source_markdown_file_history	604	Histories of confirmed GH-AW source files, including the number of observed versions
Source History	source_markdown_file_snapshot	2,820	Markdown snapshots (path, complete content, literal YAML frontmatter, and natural-language body)
	source_markdown_file_version	2,820	Ordered source-file versions (commit SHA, timestamp, rank, predecessor, and successor)
Compiled Artifacts	lock_file_snapshot	2,820	Commit-aligned compiled <code>*.lock.yml</code> snapshots (path, content, and Git blob SHA), one per source version

supporting visualizations. The corresponding links are provided below.

- Hugging Face: <https://huggingface.co/datasets/example/GHAW>
- Zenodo: <https://doi.org/10.5281/zenodo.example>
- GitHub: <https://github.com/example/GHAW>

IV. POTENTIAL RESEARCH QUESTIONS

The GHAW-H dataset provides an empirical basis for studying agentic workflow specifications. Its main arti-

facts are text-based and include natural-language instructions written in Markdown, structured frontmatter declarations, and compiled workflow definitions in YAML. This combination makes Natural Language Processing (NLP) techniques suitable for analyzing the content of these artifacts and the relationships among them. In the following, we outline example research questions that can be investigated using such techniques.

A. Instruction Structure and Developer Intent

- 1) How do GH-AW Markdown specifications express the intended behavior of an agent?
- 2) Which software engineering activities are automated through GH-AW instructions?
- 3) How do GH-AW instructions relate agent actions to the software artifacts they affect and the circumstances in which those actions should be performed?

B. Instruction Quality and Specification Defects

- 1) To what extent do GH-AW instructions contain ambiguous, incomplete, or unverifiable statements?
- 2) Which linguistic patterns are associated with unclear or underspecified agent behavior?
- 3) How do contradictions or redundant instructions affect the clarity of GH-AW specifications?

C. Instruction–Configuration Consistency

- 1) Do the permissions and tools declared in the frontmatter support the operations requested in GH-AW instructions?
- 2) Do the triggers declared in the frontmatter correspond to the events and circumstances described in the instructions?
- 3) Does the frontmatter grant permissions or tools that are not required by the intended workflow behavior?

D. Specification-to-Workflow Traceability

- 1) How are GH-AW instructions reflected in the jobs, steps, conditions, and permissions of the compiled lock file?
- 2) Which instructions are only partially represented or not represented in the compiled workflow?
- 3) Which compiled workflow elements cannot be clearly traced back to the source specification?

E. Workflow Evolution and Maintenance

- 1) How does the intended behavior expressed in GH-AW specifications evolve across versions?
- 2) Do changes in Markdown instructions correspond to changes in frontmatter configuration and compiled workflow definitions?
- 3) Which changes modify workflow behavior, and which are limited to wording or documentation?

F. Workflow Similarity and Reuse

- 1) Which GH-AW specifications express similar software engineering goals or behaviors?
- 2) To what extent do similar Markdown instructions produce similar frontmatter configurations and compiled workflows?
- 3) How can existing GH-AW specifications be retrieved and reused for new automation tasks?

V. RELATED WORK

Existing datasets on AI-based coding agents capture the different ways in which these systems operate within software repositories. AIDev collects pull requests created by agents, along with their associated commits, reviews, comments, and issues [5], whereas AgentPack focuses on code changes co-authored by humans and agents [6]. SWE-chat records real programming sessions, including user prompts, agent responses, tool calls, and code changes [7]. Other datasets collect context files, commands, skills, and subagents, for tools such as Codex, Cursor, and Gemini [8], [9]. Galster et al. [8] collect configuration files used by agentic coding tools. These datasets contain information about agent contributions, interaction trajectories, and the artifacts used to configure general-purpose coding agents. However, none of these datasets covers GH-AW or captures how its workflow specifications evolve over time.

Datasets on GitHub automation have focused primarily on the definitions and executions of conventional workflows. Cardoen et al. [10] collect histories of GitHub Actions workflow files. The dataset GHALogs complements this area with information about runs, steps, and logs [11]. Other studies have constructed run datasets to analyze resource consumption, failures, and reruns in GitHub Actions [12], [13]. These works enable the study of workflow evolution and runtime behavior, but they focus on conventional workflows or on evidence generated by their runs. In contrast, to the best of our knowledge *GHAW-H* is the first dataset focused on GH-AW.

REFERENCES

- [1] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, “Large language model-based agents for software engineering: A survey,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [2] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 50 528–50 652, 2024.
- [3] H. Li, H. Zhang, and A. E. Hassan, “The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering,” *arXiv preprint arXiv:2507.15003*, 2025.
- [4] GitHub, “GitHub Agentic Workflows are now in technical preview,” <https://github.blog/changelog/2026-02-13-github-agentic-workflows-are-now-in-technical-preview/>, Feb. 2026, GitHub Changelog. Accessed: 2026-06-16.
- [5] H. Li, H. Zhang, and A. E. Hassan, “Aidev: studying ai coding agents on github,” *arXiv preprint arXiv:2602.09185*, 2026.
- [6] Y. Zi, Z. Wu, A. Boruch-Gruszecki, J. Bell, and A. Guha, “Agentpack: A dataset of code changes, co-authored by agents and humans,” *arXiv preprint arXiv:2509.21891*, 2025.
- [7] J. Baumann, V. Padmakumar, X. Li, J. Yang, D. Yang, and S. Koyejo, “Swe-chat: Coding agent interactions from real users in the wild,” *arXiv preprint arXiv:2604.20779*, 2026.

- [8] M. Galster, S. Mohsenimofidi, L. Böhme, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "A dataset of agentic ai coding tool configurations," in *Proceedings of the 3rd ACM International Conference on AI-Powered Software*, 2026, pp. 314–322.
- [9] M. Galster, S. Mohsenimofidi, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "Configuring agentic ai coding tools: An exploratory study," 2026.
- [10] G. Cardoen, T. Mens, and A. Decan, "A dataset of github actions workflow histories," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 677–681.
- [11] F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon, "Ghalogs: Large-scale dataset of github actions runs," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 2025, pp. 669–673.
- [12] L. Zheng, S. Li, X. Huang, J. Huang, B. Lin, J. Chen, and J. Xuan, "Why do github actions workflows fail? an empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 5, pp. 1–29, 2026.
- [13] J. Huang and B. Lin, "On the reruns of github actions workflows," *ACM Transactions on Software Engineering and Methodology*, 2026.